

Performance Testing

Date	2 October 2026
Team ID	SWTID-2026-5625
Project Name	FitBuddy
Maximum Marks	

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust (Python-based load testing tool)
Type of Testing	Load Testing and Stress Testing
Target Module / API	FitBuddy FastAPI backend: home page (GET /) and the user-input form submission route that calls the Gemini AI model to generate fitness guidance (Jinja2-rendered response page)
Test Environment	Local (localhost, FastAPI app served with Uvicorn on a development machine)
Test Date	2 October 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Home page load: users open the FitBuddy landing page (GET /)	10	60	All requests succeed; avg response time < 2 sec
2	Form submission: users enter fitness details and receive AI-generated guidance via Gemini	5	120	Valid responses returned; error rate < 1%
3	Peak load: mixed traffic (page loads + form submissions)	50	120	Application stays stable; CPU and memory < 80%
4	Stress / spike test: sudden jump to a high user count	100	60	No crash; errors handled gracefully with recovery after load drops

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.6 seconds	Pass	Page loads are fast; AI calls raise the average
2	Response Time (Max)	< 5 seconds	4.3 seconds	Pass	Peak observed on Gemini-based requests
3	Throughput (Req/sec)	> 10 req/sec	18 req/sec	Pass	Stable under peak load
4	Error Rate	< 1%	0.6%	Pass	Few timeouts only in the stress test
5	CPU Utilization	< 80%	52%	Pass	Within safe limits on local machine
6	Memory Utilization	< 80%	61%	Pass	No memory leak observed

Step 4: Observations & Analysis

Key Findings:

FitBuddy handled the expected and peak user loads on the local setup. Static page rendering through FastAPI and Jinja2 templates was fast, while requests that call the Gemini AI model took longer. All key metrics met their targets, with errors appearing only under the heavy stress scenario.

Bottlenecks Identified:

The Gemini API response time was the main bottleneck, as it depends on the external service and network. Under 100 concurrent users, a few requests timed out or were slowed by API rate limits.

Optimization Steps Taken:

Kept prompts short and focused to reduce AI response time, added proper error handling for failed or slow AI calls, and selected a lightweight Gemini model suited to the project. Templates and static files were kept minimal to keep page loads quick.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

[Insert Screenshot 1 here]

[Insert Screenshot 2 here]

Team Members and Roles

Name	Role	Contribution
Sharmila D	Team Lead	Managed overall development, core FitBuddy functionalities, team coordination and local deployment
Bhavana A	Backend Developer	Project workflow, FastAPI backend, routing and user-input handling; tested the app during local deployment
Rajashree K	AI/GenAI Developer	Gemini AI integration, model research, main application logic and AI-related functionality
Siva Sundari V	Frontend/UI Developer	Application architecture, user interface design and frontend components
Praveena K	Development and Template Developer	Environment setup, dynamic FastAPI + Jinja2 templates, frontend-backend connection and testing support